

SEO and Metadata

Storefront Next template's built-in SEO features helps optimize your store's search and product discoverability. The storefront template provides SEO features with `hreflang` alternate links, canonical URLs, page titles, meta tags, indexing control, and structured data (JSON-LD)

Contents

Hreflang Alternate Links

Canonical URLs

Page Titles and Meta Tags (SeoMeta)

Hreflang Alternate Links

The storefront can render locale-aware `<link rel="alternate" hreflang="...">` tags in the document `<head>`. These tags are generated centrally in `src/utils/seo.ts` and returned from the root `meta` export in `src/root.tsx`.

Why It Matters

Search engines use `hreflang` to understand whether several pages are regional or language-specific versions of the same content.

For an ecommerce storefront, the alternate links feature is especially useful because:

- It helps search engines send shoppers to the right language or locale version of a product, category, or content page.
- It reduces confusion between similar pages that differ only by locale, currency, or URL prefix.
- It increases the likelihood that international shoppers land on the version of the storefront that matches their market.

How It Works

The root loader calls `buildSeoMetaDescriptors(...)`, which returns:

1. A canonical `<link rel="canonical">`
2. One `<link rel="alternate" hreflang="...">` per supported locale
3. An `x-default` alternate pointing to the site's default locale

The alternate URLs are built from the current request path plus the configured multisite URL prefix. In practice, this means:



- The current locale prefix is stripped before alternates are rebuilt, so switching locales keeps the same logical page path.
- Query parameters follow the same canonical normalization rules described in a section later.
- Each page includes a self-referencing hreflang tag pointing to itself, which Google requires for valid hreflang sets.

Example Output

```
1 <!-- When viewing the en-GB version of a page
2 <link rel="canonical" href="https://www.examp
3 <link rel="alternate" hreflang="en-GB" href="
4 <!-- self-referencing -->
5 <link rel="alternate" hreflang="fr-FR" href="
6 <link
7   rel="alternate"
8   hreflang="x-default"
9   href="https://www.example.com/global/en-GB/
10 />
```

When You Need to Change hreflang Tags

You usually don't add `hreflang` tags in route components manually. Update this centralized logic only if you change:

- The multisite URL prefix format
- How locale aliases map into URLs
- Which query parameters are considered content-bearing for canonicalization

Canonical URLs

Every page renders a `<link rel="canonical">` tag in the `<head>`. This tells search engines which URL should be treated as the preferred version of the page.

Why It Matters

Search engines often discover multiple URLs that show the same page content. Canonical URLs tell them which version should receive SEO credit.

For an ecommerce storefront, canonical URLs are useful because:

- Product and category pages often collect tracking parameters, sort parameters, and filter parameters that can create many URL variations.



How It Works

The canonical URL is built in `src/utils/canonical-url.ts`. Storefront Next adds it to the root-level SEO descriptors generated by `src/utils/seo.ts` and renders it from `src/root.tsx`.

It applies three normalizations:

1. **Allowlisted query parameters**—Only parameters that change page content are kept. Everything else (tracking params, analytics IDs, unknown params) is stripped.
2. **Sorted parameters**—Retained params are sorted alphabetically so that `?sort=price&q=jacket` and `?q=jacket&sort=price` produce the same canonical URL.
3. **Trailing slash removal**—Trailing slashes are removed from non-root paths (`/product/jacket/` → `/product/jacket`).

Query Parameter Allowlist

The canonical URL uses an **allowlist** that specifies which query parameters are kept in canonical URLs. Storefront Next strips any parameter not explicitly listed, such as tracking parameters, analytics IDs, and other non-content parameters.

The current allowlist is defined in `src/utils/canonical-url.ts`:

Parameter	Purpose	Used by
<code>q</code>	Search query	Search page
<code>offset</code>	Pagination offset	Category, Search
<code>sort</code>	Sort order	Category, Search
<code>refine</code>	Filter refinements	Category, Search
<code>pid</code>	Product variant ID	Product detail page

Adding a Custom Query Parameter

If you add a feature that uses a new query parameter to change the page's main content, add it to the allowlist so it's kept in the canonical URL and in locale alternates.



2. Add the parameter to the `CONTENT_PARAMS` set:

```
1 const CONTENT_PARAMS = new Set([
2   "q",
3   "offset",
4   "sort",
5   "refine",
6   "pid",
7   "view", // layout mode (grid/list)
8 ]);
```

3. Add a corresponding test in `src/utils/canonical-url.test.ts`:

```
1 it("preserves view param", () => {
2   expect(buildCanonicalUrl(origin, "/category
3     "https://www.example.com/category/mens?vi
4   });
5 });
```

When not to add a parameter: If the parameter doesn't change the page's main content, don't add it to the allowlist. Examples include analytics flags, modal triggers, and UI preferences.

Page Titles and Meta Tags (SeoMeta)

`SeoMeta` is a React component defined in [src/components/seo-meta/index.tsx](#). Route components render this component to add page-level `<title>` and `<meta>` tags. `SeoMeta` handles page titles, descriptions, robots directives, Open Graph tags, and X (formerly Twitter) Card tags.

Canonical and `hreflang` links are **not** rendered by `SeoMeta`. Those are generated centrally at the root level, as described above.

React hoists these tags into `<head>` automatically, so you can render the `SeoMeta` component inside a route component and the final tags still end up in the right place.

Why It Matters

The page title and meta tags help search engines understand what a page is about and how it should appear in search results or link previews.

For an ecommerce storefront, these tags are especially useful because:

[Try for free](#)

- Consistent metadata makes it easier for search engines to understand the difference between product, category, cart, account, and content pages.

What SeoMeta Renders

Concern	Tags
Title	<code><title></code>
Description	<code><meta name="description"></code>
Robots	<code><meta name="robots" content="noindex"></code> when <code>noIndex</code> is true
Open Graph	<code>og:title</code> , <code>og:description</code> , <code>og:type</code> , <code>og:url</code> , <code>og:image</code> , <code>og:site_name</code>
X Card / Twitter Card	<code>twitter:card</code> , <code>twitter:title</code> , <code>twitter:description</code> , <code>twitter:image</code>

Basic Usage

```
1 import { SeoMeta } from "@components/seo-meta"
2
3 // `SeoMeta` is a component that you render in your page
4 // Standard page – site name is appended automatically
5 // Renders: <title>Classic Jacket | Storefront
6 <SeoMeta
7   title="Classic Jacket"
8   description="A premium leather jacket with"
9   openGraph={{
10     type: "product",
11     url: "https://www.example.com/product/classic-jacket",
12     image: "https://www.example.com/images/classic-jacket.jpg"
13   }}
14 />
```

Props

[Try for free](#)

<code>rawTitle</code>	<code>boolean</code>	<code>false</code>
-----------------------	----------------------	--------------------

<code>description</code>	<code>string</code>	<code>-</code>
--------------------------	---------------------	----------------

<code>noIndex</code>	<code>boolean</code>	<code>false</code>
----------------------	----------------------	--------------------

<code>siteName</code>	<code>string</code>	<code>t('common:defaultSiteName')</code>
-----------------------	---------------------	--

<code>twitter</code>	<code>object</code>	<code>-</code>
----------------------	---------------------	----------------

<code>openGraph</code>	<code>object</code>	<code>-</code>
------------------------	---------------------	----------------

Title Modes

`SeoMeta` supports two title modes through the `title` and `rawTitle` props:

Mode	Props	<title> output	When to use
Suffixed (default)	<code>title="My Page"</code>	My Page My Storefront Next Store	Most pages: product, category, search, account, and content pages.



`title="My
Page"`

need full control, for example, the homepage that passes the full store name.

Fallback

(no title)

`My
Storefront
Next Store`

Only the site name.

Examples:

```
</> | 
1 // Suffixed (default) – most pages use this
2 <SeoMeta title="Classic Jacket" />
3 // → <title>Classic Jacket | Storefront Next:
4
5 // Raw – homepage or any page needing exact t
6 <SeoMeta rawTitle title="My Storefront Next S
7 // → <title>Storefront Next: Market Street –
8
9 // Fallback – no title provided
10 <SeoMeta />
11 // → <title>Storefront Next: Market Street</t
```

General Meta Tag Behavior

- **Title** comes from `title` and also contains the localized site name suffix unless `rawTitle` is set.
- **Description** renders `meta[name="description"]` and is reused for Open Graph and X Card descriptions.
- **Open Graph** is opt-in via the `openGraph` prop.
- **X Card / Twitter Card** is opt-in via the `twitter` prop, or auto-generated from `openGraph`.
- **Robots** only renders when `noIndex` is set for pages that should not be indexed.

Why Each Meta Tag Helps

- **Title:** Search engines use the page title as a strong signal for what the page is about, and it is often shown as the main clickable headline in search results.
- **Description:** Search engines may use the meta description as the summary snippet in search results, which can help attract clicks.
- **Open Graph:** Social platforms use these tags when a page is shared, helping product and content links look more complete and trustworthy.



engines focus on pages shoppers should actually land on.

Open Graph and X Card Behavior

When `openGraph` is provided, `SeoMeta` renders:

- `og:title`
- `og:description` when `description` is present
- `og:type` (defaults to `website`)
- `og:url` when provided
- `og:image` when provided
- `og:site_name`

When `twitter` is provided, or when `openGraph` is provided without an explicit `twitter` prop, `SeoMeta` renders:

- `twitter:card`
- `twitter:title`
- `twitter:description` when `description` is present
- `twitter:image` when an image is available

If `openGraph.image` is present and `twitter.cardType` is not set, the card defaults to `summary_large_image`; otherwise it defaults to `summary`.

Why Social Tags Matter for SEO

Open Graph and X Card tags aren't direct search ranking factors, but they're still valuable for storefront SEO.

When someone shares a storefront URL on a platform such as Facebook, LinkedIn, X, Slack, or other messaging tools, that platform's crawler usually fetches the page and reads its social metadata.

The crawler uses tags such as:

- `og:title` or `twitter:title` for the preview headline
- `og:description` or `twitter:description` for the preview summary
- `og:image` or `twitter:image` for the preview image
- `twitter:card` to decide which X card layout to use

This metadata helps the platform build a richer preview card instead of showing a plain URL.

For an ecommerce storefront, this is useful because:

- Product, category, and campaign links look more polished and trustworthy when shared.



from social, messaging, affiliate, and marketing channels.

- Consistent preview data reduces the chance that a platform picks incomplete or low-quality fallback content from the page.

Canonical URL Consistency for Social Tags

For pages that supply `openGraph.url`, pass the same absolute URL that you want search engines to treat as canonical. In most routes, this is the canonicalized `pageUrl` built in the loader.

This keeps search and social signals aligned around the same preferred URL.

Changing the Site Name

The site name appended to page titles (for example, " | My Storefront Next Store") comes from the `common.defaultSiteName` translation key. To change it, update the value in each locale's `translations.json`:

```
1 // src/locales/en-US/translations.json
2 {
3   "common": {
4     "defaultSiteName": "Your Store Name"
5   }
6 }
```

You can also override it per-route via the `siteName` prop:

```
1 <SeoMeta title="Sale" siteName="My Outlet Sto
2 // Renders: "Sale | My Outlet Store"
```

Route Patterns

The template uses two common patterns depending on whether a page is public or auth-protected.

Public pages: Include both title and description.

```
1 <SeoMeta
2   title={t("meta.title", { defaultValue: "Abo
3   description={t("meta.description", { default
4 />
```



```
1 <SeoMeta title={t("meta.title", { defaultValue
```

Adding Meta Tags to a New Page

1. Import `SeoMeta` and render it inside your route component:

```
1 import { SeoMeta } from "@components/seo-met
2
3 export default function MyNewPage() {
4   const { t } = useTranslation("myPage");
5
6   return (
7     <
8       <SeoMeta
9         title={t("meta.title", { defaultValue
10         description={t("meta.description", {
11       />
12     /* page content */
13   </>
14 );
15 }
```

2. If the page should be shared socially, pass `openGraph` and optionally `twitter`.
3. All title and description strings use i18n with `defaultValue` fallbacks. To localize them, add the corresponding keys to your translation files, for example `meta.title` and `meta.description`.

Where SeoMeta Is Used

Route	Title	Description
/ (Home)	Store name (raw)	Welcome message
/category/:id	Category name	Category page/gener description
/product/:id	Product name	Product page/short description
/search	Search query	Result cour + query



<code>/about-us</code>	About Us	Store mission description
<code>/login</code>	Sign In	Sign in prompt
<code>/cart</code>	Cart	Cart review prompt
<code>/signup</code>	Sign Up	Account creation prompt
<code>/checkout</code>	Checkout	–
<code>/order-confirmation/:orderNo</code>	Order Confirmation	–
<code>/account/*</code> (all sub-pages)	Page-specific title	–

Note on search and cart indexability: The template indexes `/search` and `/cart` by default. Some storefronts prefer to add `noIndex` to internal search results (to avoid thin/duplicate content) and to the cart page (session-specific content). Evaluate whether these pages provide value as search landing pages for your store and add `noIndex` if they do not.

Why Some Pages Use `noIndex`

Pages behind authentication (account, wishlist, orders) and transactional pages (checkout, order confirmation) use `noIndex` because:

- **Account pages aren't useful landing pages for search:** Crawlers trying to visit account, wishlist, or order pages will usually be redirected to the login page instead of seeing the actual account content.
- **Checkout pages are often empty without shopper state:** A crawler visiting checkout without an active basket will usually see an empty or unusable checkout experience, which is not helpful in search results.
- **Some pages are session-specific:** An order confirmation page or another shopper-specific page has no value for a general search audience.

If your implementation exposes any of these pages publicly (for example, shared wishlists), remove `noIndex` from those routes.

Why `noIndex` Helps

[Try for free](#)

- Keeping checkout, account, and other private flows out of search results
- Preventing thin, empty, or session-specific pages from being indexed
- Helping search engines spend more attention on high-value pages such as product, category, brand, and content pages

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)
[Tableau](#)
[Commerce Cloud](#)
[Lightning Design System](#)
[Einstein](#)
[Quip](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)
[APIs](#)
[Trailhead](#)
[Sample Apps](#)
[AppExchange](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)
[Partner Community](#)
[Blog](#)
[Salesforce Admins](#)
[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

